Time Series

About the Data

In this notebook, we will be working with 5 data sets:

- (CSV) Facebook's stock price daily throughout 2018 (obtained using the stock_analysis package).
- (CSV) Facebook's OHLC stock data from May 20, 2019 - May 24, 2019 per minute from Nasdaq.com.
- (CSV) melted stock data for Facebook from May 20, 2019 - May 24, 2019 per minute from Nasdaq.com.
- (DB) stock opening prices by the minute for Apple from May 20, 2019 - May 24, 2019 altered to have seconds in the time from Nasdaq.com.
- (DB) stock opening prices by the minute for Facebook from May 20, 2019 - May 24, 2019 from Nasdaq.com.

Setup

```
import numpy as np
import pandas as pd
fb = pd.read_csv('fb_2018.csv', index_col='date',
parse_dates=True).assign(
trading_volume=lambda x: pd.cut(x.volume, bins=3, labels=['low',
'med', 'high'])
)
fb.head()
```

```
               open    high       low   close    volume trading_volume
date
2018-01-02  177.68  181.58  177.5500  181.42  18151903            low
2018-01-03  181.88  184.78  181.3300  184.67  16886563            low
2018-01-04  184.90  186.21  184.0996  184.33  13880896            low
2018-01-05  185.59  186.90  184.9300  186.85  13574535            low
2018-01-08  187.20  188.90  186.3300  188.28  17994726            low
```

Time-based selection and filtering

Remember, when we have a DatetimeIndex , we can use datetime slicing. We can provide a range of dates. We only get three days back because the stock market is closed on the weekends:

```
fb['2018-10-11':'2018-10-15']
```

```
               open    high       low   close    volume trading_volume
date
2018-10-11  150.13  154.81  149.1600  153.35  35338901            low
```

```
2018-10-12  156.73  156.89  151.2998  153.74  25293492            low
2018-10-15  153.32  155.57  152.5500  153.52  15433521            low
```

We can select ranges of months and quarters:

```
fb.loc['2018-Q1'].equals(fb['2018-01':'2018-03'])

True
```

The first() method will give us a specified length of time from the beginning of the time series. Here, we ask for a week. January 1, 2018 was a holiday—meaning the market was closed. It was also a Monday, so the week here is only four days:

```
fb.first('1W')

C:\Users\jnrss\AppData\Local\Temp\ipykernel_23720\2655357208.py:1:
FutureWarning: first is deprecated and will be removed in a future
version. Please create a mask and filter using `.loc` instead
  fb.first('1W')

              open    high       low   close    volume trading_volume
date
2018-01-02  177.68  181.58  177.5500  181.42  18151903            low
2018-01-03  181.88  184.78  181.3300  184.67  16886563            low
2018-01-04  184.90  186.21  184.0996  184.33  13880896            low
2018-01-05  185.59  186.90  184.9300  186.85  13574535            low
```

The last() method will take from the end:

```
fb.last('1W')

C:\Users\jnrss\AppData\Local\Temp\ipykernel_23720\2477606097.py:1:
FutureWarning: last is deprecated and will be removed in a future
version. Please create a mask and filter using `.loc` instead
  fb.last('1W')

              open    high     low   close    volume trading_volume
date
2018-12-31  134.45  134.64  129.95  131.09  24625308            low
```

For the next few examples, we need datetimes, so we will read in the stock data per minute file:

```
stock_data_per_minute = pd.read_csv(
 'fb_week_of_may_20_per_minute.csv', index_col='date',
parse_dates=True,
 date_parser=lambda x: pd.to_datetime(x, format='%Y-%m-%d %H-%M')
)
stock_data_per_minute.head()
```

```
C:\Users\jnrss\AppData\Local\Temp\ipykernel_23720\1785798962.py:1:
FutureWarning: The argument 'date_parser' is deprecated and will be
removed in a future version. Please use 'date_format' instead, or read
your data in as 'object' dtype and then call 'to_datetime'.
  stock_data_per_minute = pd.read_csv(
```

```
                          open      high       low     close    volume
date
2019-05-20 09:30:00  181.6200  181.6200  181.6200  181.6200  159049.0
2019-05-20 09:31:00  182.6100  182.6100  182.6100  182.6100  468017.0
2019-05-20 09:32:00  182.7458  182.7458  182.7458  182.7458   97258.0
2019-05-20 09:33:00  182.9500  182.9500  182.9500  182.9500   43961.0
2019-05-20 09:34:00  183.0600  183.0600  183.0600  183.0600   79562.0
```

We can use the Grouper to roll up our data to the daily level along with first and last

```
stock_data_per_minute.groupby(pd.Grouper(freq='1D')).agg({
'open': 'first',
'high': 'max',
'low': 'min',
'close': 'last',
'volume': 'sum'
})
```

```
              open      high       low   close      volume
date
2019-05-20  181.62  184.1800  181.6200  182.72  10044838.0
2019-05-21  184.53  185.5800  183.9700  184.82   7198405.0
2019-05-22  184.81  186.5603  184.0120  185.32   8412433.0
2019-05-23  182.50  183.7300  179.7559  180.87  12479171.0
2019-05-24  182.33  183.5227  181.0400  181.06   7686030.0
```

The at_time() method allows us to pull out all datetimes that match a certain time. Here, we can grab all the rows from the time the stock market opens (930 AM):

```
stock_data_per_minute.at_time('9:30')
```

```
                       open    high     low   close    volume
date
2019-05-20 09:30:00  181.62  181.62  181.62  181.62  159049.0
2019-05-21 09:30:00  184.53  184.53  184.53  184.53   58171.0
2019-05-22 09:30:00  184.81  184.81  184.81  184.81   41585.0
2019-05-23 09:30:00  182.50  182.50  182.50  182.50  121930.0
2019-05-24 09:30:00  182.33  182.33  182.33  182.33   52681.0
```

We can use between_time() to grab data for the last two minutes of trading daily:

```
stock_data_per_minute.between_time('15:59', '16:00')
```

```
                        open     high      low    close     volume
date
2019-05-20 15:59:00  182.915  182.915  182.915  182.915   134569.0
2019-05-20 16:00:00  182.720  182.720  182.720  182.720  1113672.0
2019-05-21 15:59:00  184.840  184.840  184.840  184.840    61606.0
2019-05-21 16:00:00  184.820  184.820  184.820  184.820   801080.0
2019-05-22 15:59:00  185.290  185.290  185.290  185.290    96099.0
2019-05-22 16:00:00  185.320  185.320  185.320  185.320  1220993.0
2019-05-23 15:59:00  180.720  180.720  180.720  180.720   109648.0
2019-05-23 16:00:00  180.870  180.870  180.870  180.870  1329217.0
2019-05-24 15:59:00  181.070  181.070  181.070  181.070    52994.0
2019-05-24 16:00:00  181.060  181.060  181.060  181.060   764906.0
```

On average, are more shares traded within the first 30 minutes of trading or in the last 30 minutes? We can combine between_time() with Groupers and filter() from the aggregation.ipynb notebook to answer this question. For the week in question, more are traded on average around opening time than closing time:

```
shares_traded_in_first_30_min = stock_data_per_minute\
    .between_time('9:30', '10:00')\
    .groupby(pd.Grouper(freq='1D'))\
    .filter(lambda x: (x.volume > 0).all())\
    .volume.mean()
shares_traded_in_last_30_min = stock_data_per_minute\
    .between_time('15:30', '16:00')\
    .groupby(pd.Grouper(freq='1D'))\
    .filter(lambda x: (x.volume > 0).all())\
    .volume.mean()
shares_traded_in_first_30_min - shares_traded_in_last_30_min
```

```
18592.967741935485
```

In cases where time doesn't matter, we can normalize the times to midnight:

```
pd.DataFrame(
dict(before=stock_data_per_minute.index,
after=stock_data_per_minute.index.normalize())
).head()
```

```
               before      after
0 2019-05-20 09:30:00 2019-05-20
1 2019-05-20 09:31:00 2019-05-20
2 2019-05-20 09:32:00 2019-05-20
3 2019-05-20 09:33:00 2019-05-20
4 2019-05-20 09:34:00 2019-05-20
```

Note that we can also use normalize() on a Series object after accessing the dt attribute:

```
stock_data_per_minute.index.to_series().dt.normalize().head()
```

```
date
2019-05-20 09:30:00   2019-05-20
2019-05-20 09:31:00   2019-05-20
2019-05-20 09:32:00   2019-05-20
2019-05-20 09:33:00   2019-05-20
2019-05-20 09:34:00   2019-05-20
Name: date, dtype: datetime64[ns]
```

Shifting for lagged data

We can use shift() to create some lagged data. By default, the shift will be one period. For example, we can use shift() to create a new column that indicates the previous day's closing price. From this new column, we can calculate the price change due to after hours trading (after the close one day right up to the open the following day):

```
fb.assign(
prior_close=lambda x: x.close.shift(),
after_hours_change_in_price=lambda x: x.open - x.prior_close,
abs_change=lambda x: x.after_hours_change_in_price.abs()
).nlargest(5, 'abs_change')

               open    high     low   close     volume
trading_volume  \
date

2018-07-26  174.89  180.13  173.75  176.26  169803668           high

2018-04-26  173.22  176.27  170.80  174.16   77556934            med

2018-01-12  178.06  181.48  177.40  179.37   77551299            med

2018-10-31  155.00  156.40  148.96  151.79   60101251            low

2018-03-19  177.01  177.17  170.06  172.56   88140060            med


            prior_close  after_hours_change_in_price  abs_change
date
2018-07-26       217.50                       -42.61       42.61
2018-04-26       159.69                        13.53       13.53
2018-01-12       187.77                        -9.71        9.71
2018-10-31       146.22                         8.78        8.78
2018-03-19       185.09                        -8.08        8.08
```

The tshift() method will shift the DatetimeIndex rather than the data. However, if the goal is to to add/subtract time we can use pd.Timedelta :

```
pd.date_range('2018-01-01', freq='D', periods=5) + pd.Timedelta('9
hours 30 minutes')
```

```
DatetimeIndex(['2018-01-01 09:30:00', '2018-01-02 09:30:00',
               '2018-01-03 09:30:00', '2018-01-04 09:30:00',
               '2018-01-05 09:30:00'],
              dtype='datetime64[ns]', freq='D')
```

When working with stock data, we only have data for the dates the market was open. We can use first_valid_index() to give us the index of the first non-null entry in our data. For September 2018, this is September 4th:

```
fb.index = pd.to_datetime(fb.index)
first_valid = fb.loc['2018-09'].first_valid_index()
print(first_valid)
```

```
2018-09-04 00:00:00
```

Conversely, we can use last_valid_index() to get the last entry of non-null data. For September 2018, this is September 28th:

```
last_valid = fb.loc['2018-09'].last_valid_index()
print(last_valid)
```

```
2018-09-28 00:00:00
```

We can use asof() to find the last non-null data before the point we are looking for, if it isn't in the index. From the previous result, we know that the market was not open on September 30th. It also isn't in the index:

```
fb.index.contains('2018-09-30')
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[158], line 1
----> 1 fb.index.contains('2018-09-30')

AttributeError: 'DatetimeIndex' object has no attribute 'contains'
```

```
exists = fb.index.isin([pd.Timestamp('2018-09-30')])
print(exists.any())
```

If we ask for it, we will get the data from the index we got from fb['2018-09'].last_valid_index() , which was September 28th:

```
fb.asof('2018-09-30')
```

```
open                 168.33
high                 168.79
low                  162.56
```

```
close                 164.46
volume              34265638
trading_volume           low
Name: 2018-09-30 00:00:00, dtype: object
```

Differenced data

Using the diff() method is a quick way to calculate the difference between the data and a lagged version of it. By default, it will yield the result of data - data.shift() :

```
(
fb.drop(columns='trading_volume')
    - fb.drop(columns='trading_volume').shift()
).equals(
    fb.drop(columns='trading_volume').diff()
)

True
```

We can use this to see how Facebook stock changed day-over-day:

```
fb.drop(columns='trading_volume').diff().head()

            open  high     low  close     volume
date
2018-01-02   NaN   NaN     NaN    NaN        NaN
2018-01-03  4.20  3.20  3.7800   3.25 -1265340.0
2018-01-04  3.02  1.43  2.7696  -0.34 -3005667.0
2018-01-05  0.69  0.69  0.8304   2.52  -306361.0
2018-01-08  1.61  2.00  1.4000   1.43  4420191.0
```

We can specify the number of periods, can be any positive or negative integer:m

```
fb.drop(columns='trading_volume').diff(-3).head()

            open  high     low  close     volume
date
2018-01-02 -7.91 -5.32 -7.3800  -5.43  4577368.0
2018-01-03 -5.32 -4.12 -5.0000  -3.61 -1108163.0
2018-01-04 -3.80 -2.59 -3.0004  -3.54  1487839.0
2018-01-05 -1.35 -0.99 -0.7000  -0.99  3044641.0
2018-01-08 -1.20  0.50 -1.0500   0.51  8406139.0
```

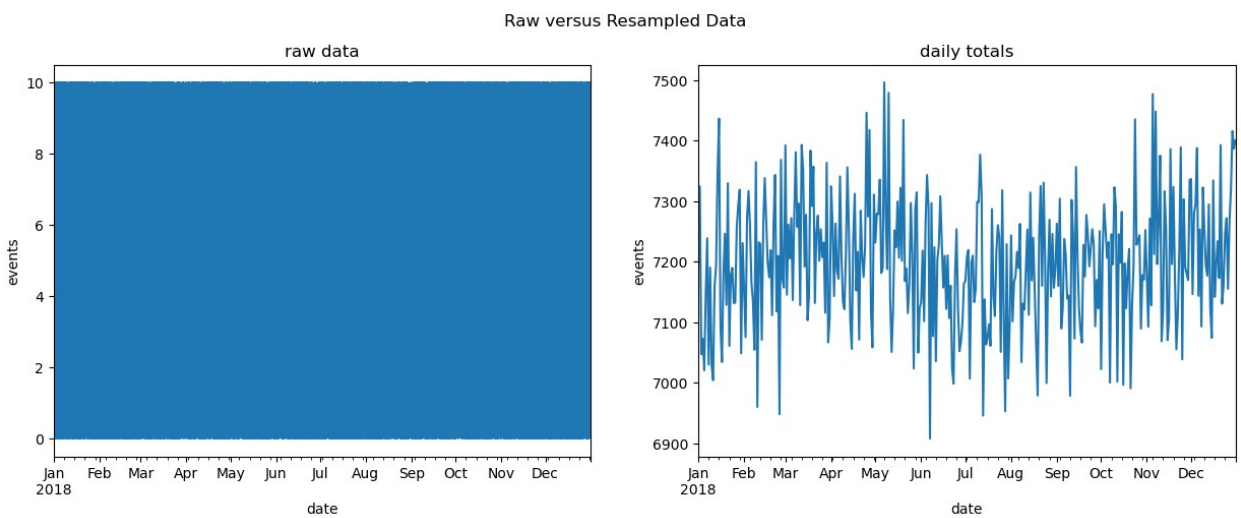
Resampling

Sometimes the data is at a granularity that isn't conducive to our analysis. Consider the case where we have data per minute for the full year of 2018. Let's see what happens if we try to plot this. Plotting will be covered in the next module, so don't worry too much about the code. First, we import matplotlib for plotting:

```
import matplotlib.pyplot as plt
```

Then we will look at the plot at the minute level and at the daily aggregated level (summed):

```
np.random.seed(0)
index = pd.date_range('2018-01-01', freq='min', periods=365*24*60)
raw = pd.DataFrame(
    np.random.uniform(0, 10, size=index.shape[0]), index=index
)
fig, axes = plt.subplots(1, 2, figsize=(15, 5))
raw.plot(legend=False, ax=axes[0], title='raw data')
raw.resample('1D').sum().plot(legend=False, ax=axes[1], title='daily
totals')
for ax in axes:
    ax.set_xlabel('date')
    ax.set_ylabel('events')
plt.suptitle('Raw versus Resampled Data')
plt.show()
```



The plot on the left has so much data we can't see anything. However, when we aggregate to the daily totals, we see the data. We can alter the granularity of the data we are working with using resampling. Recall our minute-by-minute stock data:

```
stock_data_per_minute.head()
```

	open	high	low	close	volume
date					
2019-05-20 09:30:00	181.6200	181.6200	181.6200	181.6200	159049.0
2019-05-20 09:31:00	182.6100	182.6100	182.6100	182.6100	468017.0
2019-05-20 09:32:00	182.7458	182.7458	182.7458	182.7458	97258.0
2019-05-20 09:33:00	182.9500	182.9500	182.9500	182.9500	43961.0
2019-05-20 09:34:00	183.0600	183.0600	183.0600	183.0600	79562.0

We can resample this to get to a daily frequency:

```
stock_data_per_minute.resample('1D').agg({
'open': 'first',
'high': 'max',
'low': 'min',
'close': 'last',
'volume': 'sum'
})
```

```
               open      high       low   close      volume
date
2019-05-20  181.62  184.1800  181.6200  182.72  10044838.0
2019-05-21  184.53  185.5800  183.9700  184.82   7198405.0
2019-05-22  184.81  186.5603  184.0120  185.32   8412433.0
2019-05-23  182.50  183.7300  179.7559  180.87  12479171.0
2019-05-24  182.33  183.5227  181.0400  181.06   7686030.0
```

We can downsample to quarterly data:

```
fb.resample('Q').mean()
```

```
C:\Users\jnrss\AppData\Local\Temp\ipykernel_23720\46532002.py:1:
FutureWarning: 'Q' is deprecated and will be removed in a future
version, please use 'QE' instead.
  fb.resample('Q').mean()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call
last)
Cell In[167], line 1
----> 1 fb.resample('Q').mean()

File D:\Program Files\Anaconda\Lib\site-packages\pandas\core\
resample.py:1384, in Resampler.mean(self, numeric_only, *args,
**kwargs)
   1382 maybe_warn_args_and_kwargs(type(self), "mean", args, kwargs)
   1383 nv.validate_resampler_func("mean", args, kwargs)
-> 1384 return self._downsample("mean", numeric_only=numeric_only)

File D:\Program Files\Anaconda\Lib\site-packages\pandas\core\
resample.py:1782, in DatetimeIndexResampler._downsample(self, how,
**kwargs)
   1779 # we are downsampling
   1780 # we want to call the actual grouper method here
   1781 if self.axis == 0:
-> 1782     result = obj.groupby(self._grouper).aggregate(how,
**kwargs)
   1783 else:
```

```
   1784     # test_resample_axis1
   1785     result = obj.T.groupby(self._grouper).aggregate(how,
**kwargs).T

File D:\Program Files\Anaconda\Lib\site-packages\pandas\core\groupby\
generic.py:1432, in DataFrameGroupBy.aggregate(self, func, engine,
engine_kwargs, *args, **kwargs)
   1429     kwargs["engine_kwargs"] = engine_kwargs
   1431 op = GroupByApply(self, func, args=args, kwargs=kwargs)
-> 1432 result = op.agg()
   1433 if not is_dict_like(func) and result is not None:
   1434     # GH #52849
   1435     if not self.as_index and is_list_like(func):

File D:\Program Files\Anaconda\Lib\site-packages\pandas\core\
apply.py:187, in Apply.agg(self)
    184 kwargs = self.kwargs
    186 if isinstance(func, str):
--> 187     return self.apply_str()
    189 if is_dict_like(func):
    190     return self.agg_dict_like()

File D:\Program Files\Anaconda\Lib\site-packages\pandas\core\
apply.py:603, in Apply.apply_str(self)
    601         else:
    602             self.kwargs["axis"] = self.axis
--> 603 return self._apply_str(obj, func, *self.args, **self.kwargs)

File D:\Program Files\Anaconda\Lib\site-packages\pandas\core\
apply.py:693, in Apply._apply_str(self, obj, func, *args, **kwargs)
    691 f = getattr(obj, func)
    692 if callable(f):
--> 693     return f(*args, **kwargs)
    695 # people may aggregate on a non-callable attribute
    696 # but don't let them think they can pass args to it
    697 assert len(args) == 0

File D:\Program Files\Anaconda\Lib\site-packages\pandas\core\groupby\
groupby.py:2452, in GroupBy.mean(self, numeric_only, engine,
engine_kwargs)
   2445     return self._numba_agg_general(
   2446         grouped_mean,
   2447         executor.float_dtype_mapping,
   2448         engine_kwargs,
   2449         min_periods=0,
   2450     )
   2451 else:
-> 2452     result = self._cython_agg_general(
   2453         "mean",
   2454         alt=lambda x: Series(x,
```

```
copy=False).mean(numeric_only=numeric_only),
   2455         numeric_only=numeric_only,
   2456     )
   2457     return result.__finalize__(self.obj, method="groupby")

File D:\Program Files\Anaconda\Lib\site-packages\pandas\core\groupby\
groupby.py:1998, in GroupBy._cython_agg_general(self, how, alt,
numeric_only, min_count, **kwargs)
   1995     result = self._agg_py_fallback(how, values,
ndim=data.ndim, alt=alt)
   1996     return result
-> 1998 new_mgr = data.grouped_reduce(array_func)
   1999 res = self._wrap_agged_manager(new_mgr)
   2000 if how in ["idxmin", "idxmax"]:

File D:\Program Files\Anaconda\Lib\site-packages\pandas\core\
internals\managers.py:1472, in BlockManager.grouped_reduce(self, func)
   1470             result_blocks = extend_blocks(applied,
result_blocks)
   1471     else:
-> 1472         applied = blk.apply(func)
   1473         result_blocks = extend_blocks(applied, result_blocks)
   1475 if len(result_blocks) == 0:

File D:\Program Files\Anaconda\Lib\site-packages\pandas\core\
internals\blocks.py:393, in Block.apply(self, func, **kwargs)
    387 @final
    388 def apply(self, func, **kwargs) -> list[Block]:
    389     """
    390     apply the function to my values; return a block if we are
not
    391     one
    392     """
--> 393     result = func(self.values, **kwargs)
    395     result = maybe_coerce_values(result)
    396     return self._split_op_result(result)

File D:\Program Files\Anaconda\Lib\site-packages\pandas\core\groupby\
groupby.py:1973, in
GroupBy._cython_agg_general.<locals>.array_func(values)
   1971 def array_func(values: ArrayLike) -> ArrayLike:
   1972     try:
-> 1973         result = self._grouper._cython_operation(
   1974             "aggregate",
   1975             values,
   1976             how,
   1977             axis=data.ndim - 1,
   1978             min_count=min_count,
   1979             **kwargs,
   1980         )
```

```
   1981     except NotImplementedError:
   1982         # generally if we have numeric_only=False
   1983         # and non-applicable functions
   1984         # try to python agg
   1985         # TODO: shouldn't min_count matter?
   1986         # TODO: avoid special casing SparseArray here
   1987         if how in ["any", "all"] and isinstance(values,
SparseArray):

File D:\Program Files\Anaconda\Lib\site-packages\pandas\core\groupby\
ops.py:831, in BaseGrouper._cython_operation(self, kind, values, how,
axis, min_count, **kwargs)
    829 ids, _, _ = self.group_info
    830 ngroups = self.ngroups
--> 831 return cy_op.cython_operation(
    832     values=values,
    833     axis=axis,
    834     min_count=min_count,
    835     comp_ids=ids,
    836     ngroups=ngroups,
    837     **kwargs,
    838 )

File D:\Program Files\Anaconda\Lib\site-packages\pandas\core\groupby\
ops.py:541, in WrappedCythonOp.cython_operation(self, values, axis,
min_count, comp_ids, ngroups, **kwargs)
    537 self._validate_axis(axis, values)
    539 if not isinstance(values, np.ndarray):
    540     # i.e. ExtensionArray
--> 541     return values._groupby_op(
    542         how=self.how,
    543         has_dropped_na=self.has_dropped_na,
    544         min_count=min_count,
    545         ngroups=ngroups,
    546         ids=comp_ids,
    547         **kwargs,
    548     )
    550 return self._cython_op_ndim_compat(
    551     values,
    552     min_count=min_count,
   (...)
    556     **kwargs,
    557 )

File D:\Program Files\Anaconda\Lib\site-packages\pandas\core\arrays\
categorical.py:2740, in Categorical._groupby_op(self, how,
has_dropped_na, min_count, ngroups, ids, **kwargs)
   2738     if kind == "transform":
   2739         raise TypeError(f"{dtype} type does not support {how}
operations")
```

```
-> 2740     raise TypeError(f"{dtype} dtype does not support
aggregation '{how}'")
   2742 result_mask = None
   2743 mask = self.isna()

TypeError: category dtype does not support aggregation 'mean'
```

```
#the original instruction did not work so I searched for an
alternative
fb_numeric = fb.select_dtypes(include=['number'])

# Resample and apply mean to the numeric columns
fb_resampled = fb_numeric.resample('QE').mean()
fb_resampled
```

```
                   open        high         low       close
volume
date

2018-03-31  179.472295  181.794659  177.040428  179.551148
3.292640e+07
2018-06-30  180.373770  182.277689  178.595964  180.704688
2.405532e+07
2018-09-30  180.812130  182.890886  178.955229  181.028492
2.701982e+07
2018-12-31  145.272460  147.620121  142.718943  144.868730
2.697433e+07
```

We can also use apply() . Here, we show the quarterly change from start to end:

```
fb.drop(columns='trading_volume').resample('QE').apply(
    lambda x: x.loc[x.index[-1]] - x.loc[x.index[0]]
)
```

```
              open     high     low  close    volume
date
2018-03-31 -22.53 -20.1600 -23.410 -21.63  41282390
2018-06-30  39.51  38.3997  39.844  38.93 -20984389
2018-09-30 -25.04 -28.6600 -29.660 -32.90  20304060
2018-12-31 -28.58 -31.2400 -31.310 -31.35  -1782369
```

Consider the following melted stock data by the minute. We don't see the OHLC data directly:

```
melted_stock_data = pd.read_csv('melted_stock_data.csv',
index_col='date', parse_dates=True)
melted_stock_data.head()
```

```
                        price
date
2019-05-20 09:30:00  181.6200
```

```
2019-05-20 09:31:00  182.6100
2019-05-20 09:32:00  182.7458
2019-05-20 09:33:00  182.9500
2019-05-20 09:34:00  183.0600
```

We can use the ohlc() method after resampling to recover the OHLC columns:

```
melted_stock_data.resample('1D').ohlc()['price']
```

```
               open      high       low   close
date
2019-05-20  181.62  184.1800  181.6200  182.72
2019-05-21  184.53  185.5800  183.9700  184.82
2019-05-22  184.81  186.5603  184.0120  185.32
2019-05-23  182.50  183.7300  179.7559  180.87
2019-05-24  182.33  183.5227  181.0400  181.06
```

Alternatively, we can upsample to increase the granularity. Note this will introduce NaN values:

```
fb.resample('6h').asfreq().head()
```

```
                        open    high     low   close      volume
trading_volume
date

2018-01-02 00:00:00  177.68  181.58  177.55  181.42  18151903.0
low
2018-01-02 06:00:00     NaN     NaN     NaN     NaN         NaN
NaN
2018-01-02 12:00:00     NaN     NaN     NaN     NaN         NaN
NaN
2018-01-02 18:00:00     NaN     NaN     NaN     NaN         NaN
NaN
2018-01-03 00:00:00  181.88  184.78  181.33  184.67  16886563.0
low
```

There are many ways to handle these NaN values. We can forward-fill with pad()

```
fb.resample('6h').ffill().head() #since pad is not is not  available
for resampling in newer versions of pandas
```

```
                        open    high     low   close    volume
trading_volume
date

2018-01-02 00:00:00  177.68  181.58  177.55  181.42  18151903
low
2018-01-02 06:00:00  177.68  181.58  177.55  181.42  18151903
low
```

```
2018-01-02 12:00:00  177.68  181.58  177.55  181.42  18151903
low
2018-01-02 18:00:00  177.68  181.58  177.55  181.42  18151903
low
2018-01-03 00:00:00  181.88  184.78  181.33  184.67  16886563
low
```

We can specify a specific value or a method with fillna() :

```
fb.resample('6h').fillna('nearest').head()
```

```
C:\Users\jnrss\AppData\Local\Temp\ipykernel_23720\1185989255.py:1:
FutureWarning: DatetimeIndexResampler.fillna is deprecated and will be
removed in a future version. Use obj.ffill(), obj.bfill(), or
obj.nearest() instead.
  fb.resample('6h').fillna('nearest').head()
```

```
                        open    high     low   close    volume
trading_volume
date

2018-01-02 00:00:00  177.68  181.58  177.55  181.42  18151903
low
2018-01-02 06:00:00  177.68  181.58  177.55  181.42  18151903
low
2018-01-02 12:00:00  181.88  184.78  181.33  184.67  16886563
low
2018-01-02 18:00:00  181.88  184.78  181.33  184.67  16886563
low
2018-01-03 00:00:00  181.88  184.78  181.33  184.67  16886563
low
```

We can use asfreq() and assign() to specify the action per column:

```
fb.resample('6H').asfreq().assign(
volume=lambda x: x.volume.fillna(0), # put 0 when market is closed
close=lambda x: x.close.fillna(method='ffill'), # carry forward
# take the closing price if these aren't available
open=lambda x: np.where(x.open.isnull(), x.close, x.open),
high=lambda x: np.where(x.high.isnull(), x.close, x.high),
low=lambda x: np.where(x.low.isnull(), x.close, x.low)
).head()
```

```
C:\Users\jnrss\AppData\Local\Temp\ipykernel_23720\3818699665.py:1:
FutureWarning: 'H' is deprecated and will be removed in a future
version, please use 'h' instead.
  fb.resample('6H').asfreq().assign(
C:\Users\jnrss\AppData\Local\Temp\ipykernel_23720\3818699665.py:3:
FutureWarning: Series.fillna with 'method' is deprecated and will
```

```
raise in a future version. Use obj.ffill() or obj.bfill() instead.
  close=lambda x: x.close.fillna(method='ffill'), # carry forward
```

```
                        open    high     low   close      volume
trading_volume
date

2018-01-02 00:00:00  177.68  181.58  177.55  181.42  18151903.0
low
2018-01-02 06:00:00  181.42  181.42  181.42  181.42         0.0
NaN
2018-01-02 12:00:00  181.42  181.42  181.42  181.42         0.0
NaN
2018-01-02 18:00:00  181.42  181.42  181.42  181.42         0.0
NaN
2018-01-03 00:00:00  181.88  184.78  181.33  184.67  16886563.0
low
```

Merging

We saw merging examples the querying_and_merging notebook. However, they all matched based on keys. With time series, it is possible that they are so granular that we never have the same time for multiple entries. Let's work with some stock data at different granularities:

```
import sqlite3
with sqlite3.connect('stocks.db') as connection:
 fb_prices = pd.read_sql(
 'SELECT * FROM fb_prices', connection,
 index_col='date', parse_dates=['date']
 )
 aapl_prices = pd.read_sql(
 'SELECT * FROM aapl_prices', connection,
 index_col='date', parse_dates=['date']
 )
```

The Facebook prices are at the minute granularity:

```
fb_prices.index.second.unique()
```

```
Index([0], dtype='int32', name='date')
```

However, the Apple prices have information for the second:

```
aapl_prices.index.second.unique()
```

```
Index([ 0, 52, 36, 34, 55, 35,  7, 12, 59, 17,  5, 20, 26, 23, 54, 49,
19, 53,
       11, 22, 13, 21, 10, 46, 42, 38, 33, 18, 16,  9, 56, 39,  2, 50,
31, 58,
```

```
       48, 24, 29,  6, 47, 51, 40,  3, 15, 14, 25,  4, 43,  8, 32, 27,
30, 45,
        1, 44, 57, 41, 37, 28],
      dtype='int32', name='date')
```

We can perform an asof merge to try to line these up the best we can. We specify how to handle the mismatch with the direction and tolerance parameters. We will fill in with the direction of nearest and a tolerance of 30 seconds. This will place the Apple data with the minute that it is closest to, so 93152 will go with 932 and 93707 will go with 937. Since the times are on the index, we pass left_index and right_index , as we did with merges earlier this chapter:

```
pd.merge_asof(
    fb_prices, aapl_prices,
    left_index=True, right_index=True, # datetimes are in the index
    # merge with nearest minute
    direction='nearest', tolerance=pd.Timedelta(30, unit='s')
).head()
```

```
                           FB      AAPL
date
2019-05-20 09:30:00  181.6200  183.5200
2019-05-20 09:31:00  182.6100       NaN
2019-05-20 09:32:00  182.7458  182.8710
2019-05-20 09:33:00  182.9500  182.5000
2019-05-20 09:34:00  183.0600  182.1067
```

If we don't want to lose the seconds information with the Apple data, we can use pd.merge_ordered() instead, which will interleave the two. Note this is an outer join by default (how parameter). The only catch here is that we need to reset the index in order to join on it:

```
pd.merge_ordered(
    fb_prices.reset_index(), aapl_prices.reset_index()
).set_index('date').head()
```

```
                           FB     AAPL
date
2019-05-20 09:30:00  181.6200  183.520
2019-05-20 09:31:00  182.6100      NaN
2019-05-20 09:31:52       NaN  182.871
2019-05-20 09:32:00  182.7458      NaN
2019-05-20 09:32:36       NaN  182.500
```

We can pass a fill_method to handle NaN values:

```
pd.merge_ordered(
    fb_prices.reset_index(), aapl_prices.reset_index(),
    fill_method='ffill'
).set_index('date').head()
```

```
                              FB     AAPL
date
2019-05-20 09:30:00  181.6200  183.520
2019-05-20 09:31:00  182.6100  183.520
2019-05-20 09:31:52  182.6100  182.871
2019-05-20 09:32:00  182.7458  182.871
2019-05-20 09:32:36  182.7458  182.500
```

Alternatively, we can use fillna() .

```

```